SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: MLA0301

COURSE CODE: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO2: AT2_ Application - Based Questions

concepts to model decision-making problems. (BL2,BL3)

Assessment Tool Weightage: 30%

Dr G. A. SENTHIL

STUDENT.NAME:-M.GUNA SHEKAR

REG.NO:-192472345

CASE STUDY 1:

CASE STUDY 1: Ride-Sharing Driver–Passenger Matching Using Reinforcement Learning

Aim

To develop a Reinforcement Learning-based ride-sharing system that efficiently matches drivers with passengers, reducing waiting time and improving service quality.

Objective

- Minimize passenger waiting time.
- Maximize driver utilization.
- Reduce idle time.
- Improve customer satisfaction.
- Optimize driver allocation.

Problem Statement

Traditional ride-sharing systems often rely on static or heuristic matching methods that may not adapt well to changing demand. Reinforcement Learning enables the system to learn optimal driver-passenger matching policies from real-time interactions.

Software Requirements

- Python 3.x
- VS Code
- NumPy

Process

1. Collect ride requests and driver locations.
2. Observe the current state.
3. Select a driver assignment action.
4. Execute the assignment.
5. Receive a reward based on waiting time and successful pickup.
6. Update the Q-table.
7. Repeat continuously.

State Space

- Driver location
- Passenger location
- Number of available drivers
- Traffic conditions

Reward Function

- +100 → Successful quick pickup
- +50 → Reduced waiting time
- -20 → Driver idle time
- -50 → Passenger cancellation

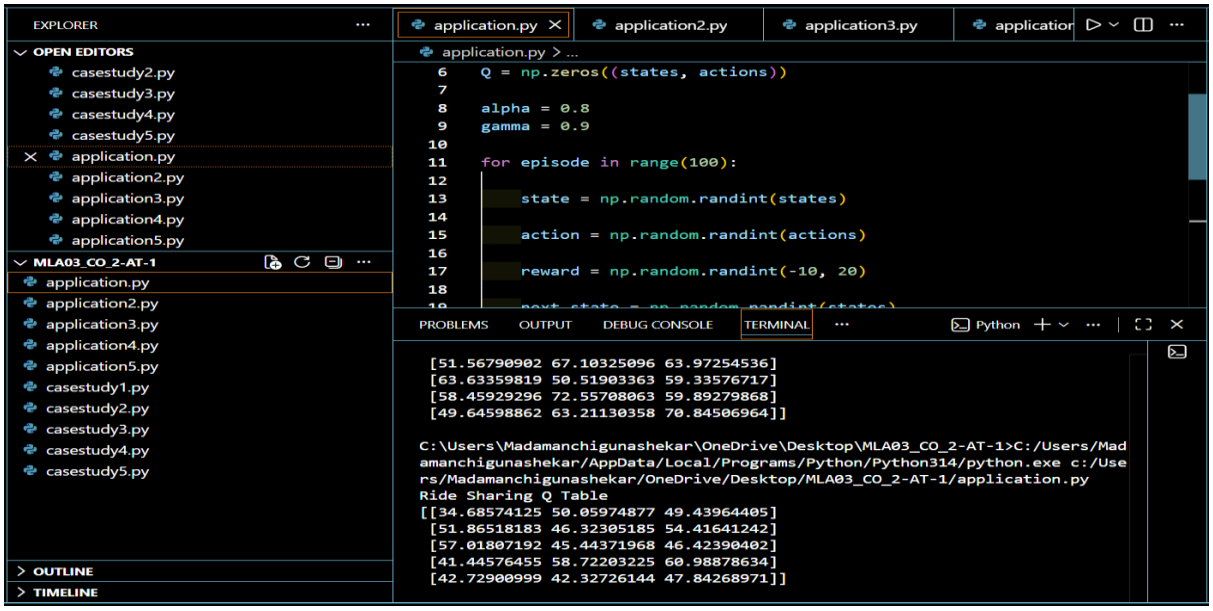
Algorithm

1. Initialize the Q-table.
2. Observe the current state.
3. Choose an action using the ε-greedy policy.
4. Execute the action.
5. Receive reward.
6. Update the Q-value.
7. Repeat until the policy converges.

Expected Output

- Faster driver-passenger matching
- Reduced waiting time
- Improved service quality

CODE:-



Conclusion

RL enables ride-sharing platforms to continuously improve matching efficiency based on demand and traffic patterns.

CASE STUDY 2: Personalized E-Learning Recommendation Using Reinforcement Learning

Aim

To develop an RL-based recommendation system that suggests personalized study materials according to learner behaviour.

Objective

- Recommend suitable learning content.
- Balance familiar and new topics.
- Increase learning engagement.
- Improve course completion rates.

Problem Statement

Static recommendation systems may repeatedly suggest similar content and fail to adapt to changing learner interests. Reinforcement Learning balances exploration of new materials with exploitation of known preferences.

Software Requirements

- Python
- NumPy

Process

1. Collect learner history.
2. Observe learner state.
3. Recommend study material.
4. Collect learner feedback.
5. Calculate reward.
6. Update the policy.
7. Repeat.

State Space

- Learning progress
- Quiz scores
- Completed lessons
- Interests

Algorithm

1. Initialize the Q-table.
2. Observe learner state.
3. Select recommendation.

4. Receive learner feedback.
5. Update Q-values.
6. Repeat.

Expected Output

- Personalized recommendations
- Improved learning outcomes
- Increased engagement

Applications

- Coursera
- Udemy
- BYJU'S
- Khan Academy

CODE:-

Conclusion

Reinforcement Learning continuously improves recommendations by learning from student interactions and feedback.

CASE STUDY 3: Robotic Vacuum Cleaner Using Reinforcement Learning

Aim

To develop an RL-based robotic vacuum cleaner that efficiently cleans rooms while avoiding obstacles.

Objective

- Maximize cleaning coverage.
- Avoid collisions.
- Reduce cleaning time.
- Optimize battery usage.

Problem Statement

Rule-based robotic cleaners follow fixed paths and may repeatedly clean the same areas or collide with obstacles. Reinforcement Learning enables the robot to learn efficient cleaning policies.

Software Requirements

- Python
- NumPy

Process

1. Observe the room environment.
2. Detect dirt and obstacles.
3. Select a movement action.
4. Execute the action.
5. Receive a reward.
6. Update the policy.
7. Continue until cleaning is complete.

State Space

- Robot position
- Dirt locations
- Obstacle positions
- Battery level

Action Space

- Move forward
- Turn left
- Turn right
- Clean

Reward Function

- +50 → Cleaned dirt
- +20 → Efficient movement
- -30 → Collision
- -10 → Unnecessary movement

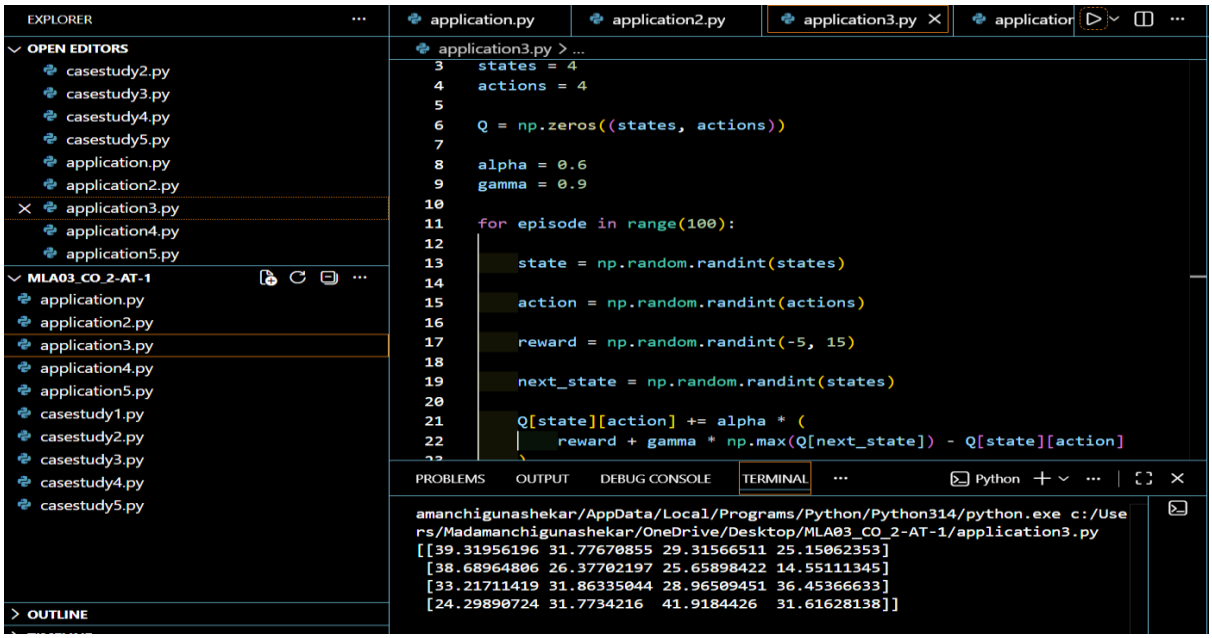
Algorithm

1. Initialize the Q-table.
2. Observe the environment.
3. Select an action using the ε-greedy policy.
4. Execute the action.
5. Receive reward.
6. Update the Q-value.
7. Repeat until all areas are cleaned.

Expected Output

- Efficient room cleaning
- Fewer collisions
- Lower energy consumption

CODE:-



Conclusion

An RL-based robotic vacuum cleaner continuously improves its cleaning strategy, resulting in higher efficiency and better obstacle avoidance.

CASE STUDY 4: Cryptocurrency Trading Using Reinforcement Learning

Aim

To develop an RL-based trading system that decides when to buy, sell, or hold cryptocurrency to maximize long-term returns.

Objective

- Maximize profit.
- Minimize trading losses.
- Learn market trends.
- Improve investment decisions.

Problem Statement

Cryptocurrency markets are highly volatile, making fixed trading rules ineffective. Reinforcement Learning learns trading strategies from market behaviour and reward feedback.

Software Requirements

- Python
- NumPy
- Pandas

Process

1. Collect market data.
2. Observe the current market state.
3. Select an action (buy, sell, or hold).
4. Execute the trade.
5. Calculate reward.
6. Update the policy.
7. Repeat.

State Space

- Current price
- Price trend
- Trading volume
- Portfolio balance

Action Space

- Buy
- Sell
- Hold

Reward Function

- +100 → Profitable trade
- +50 → Correct hold decision
- -100 → Trading loss

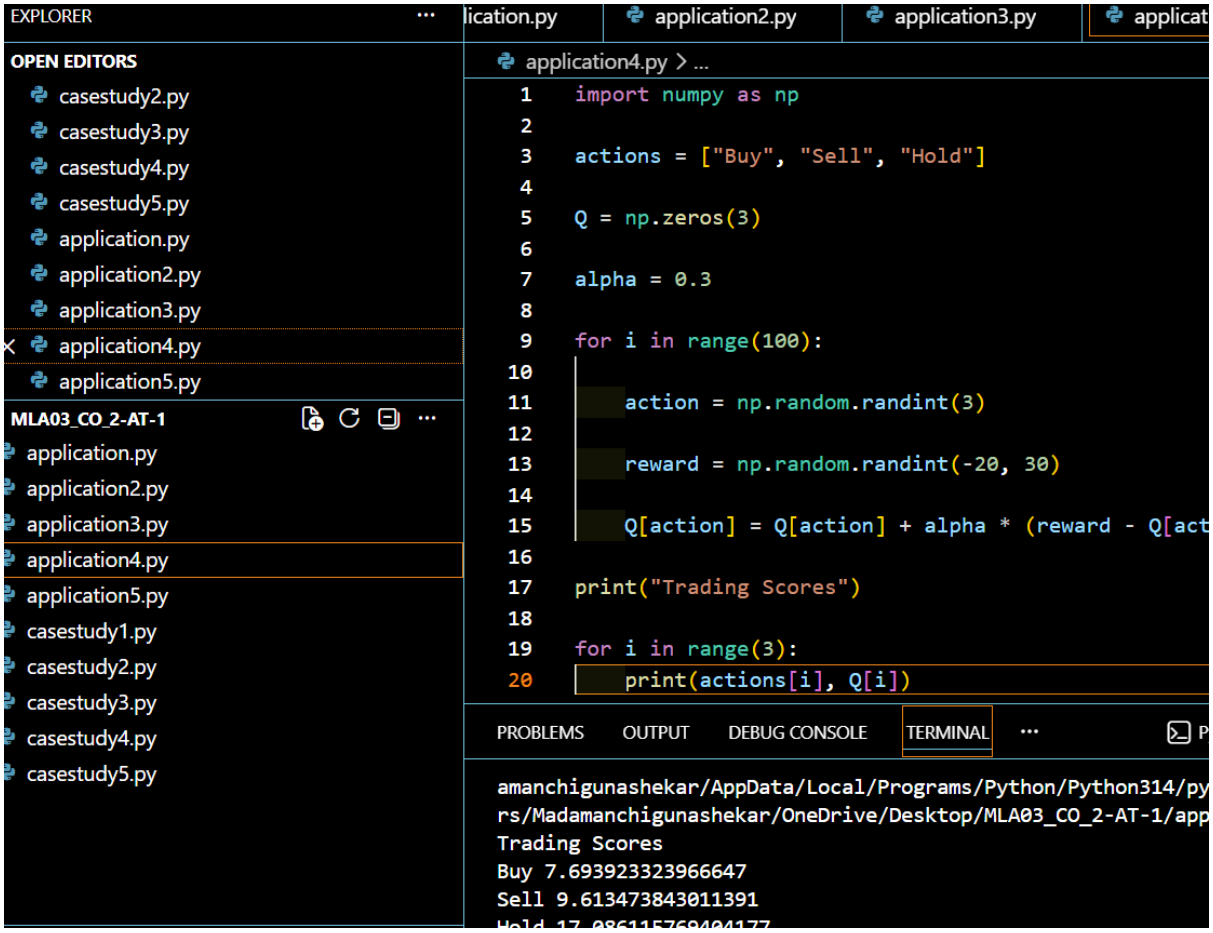
Algorithm

1. Initialize the Q-table.
2. Observe the market state.
3. Choose an action.
4. Execute the trade.
5. Receive reward.
6. Update the Q-value.
7. Repeat over multiple trading episodes.

Expected Output

- Better trading decisions
- Increased long-term profit
- Reduced trading losses

CODE:-



Conclusion

RL enables trading systems to refine their strategies over time by learning from market outcomes and reward signals.

CASE STUDY 5: Smart Grid Electricity Distribution Using Reinforcement Learning

Aim

To develop an RL-based smart grid system that optimizes electricity distribution while balancing supply and demand.

Objective

- Balance electricity demand.
- Reduce power losses.
- Improve grid reliability.
- Optimize energy distribution.

Problem Statement

Traditional electricity distribution systems use predefined control rules that may not respond efficiently to fluctuating demand. Reinforcement Learning dynamically adjusts distribution decisions based on real-time grid conditions.

Software Requirements

- Python
- NumPy
- Matplotlib

Process

1. Collect electricity demand and supply data.
2. Observe the current grid state.
3. Select a power distribution action.
4. Execute the action.
5. Receive a reward based on efficiency.
6. Update the Q-table.
7. Repeat continuously.

State Space

- Power demand
- Available generation
- Grid load
- Battery storage level

Action Space

- Increase supply
- Decrease supply

- Store excess energy

Reward Function

- +100 → Balanced supply and demand
- +50 → Reduced energy loss
- -50 → Power imbalance
- -100 → Grid overload

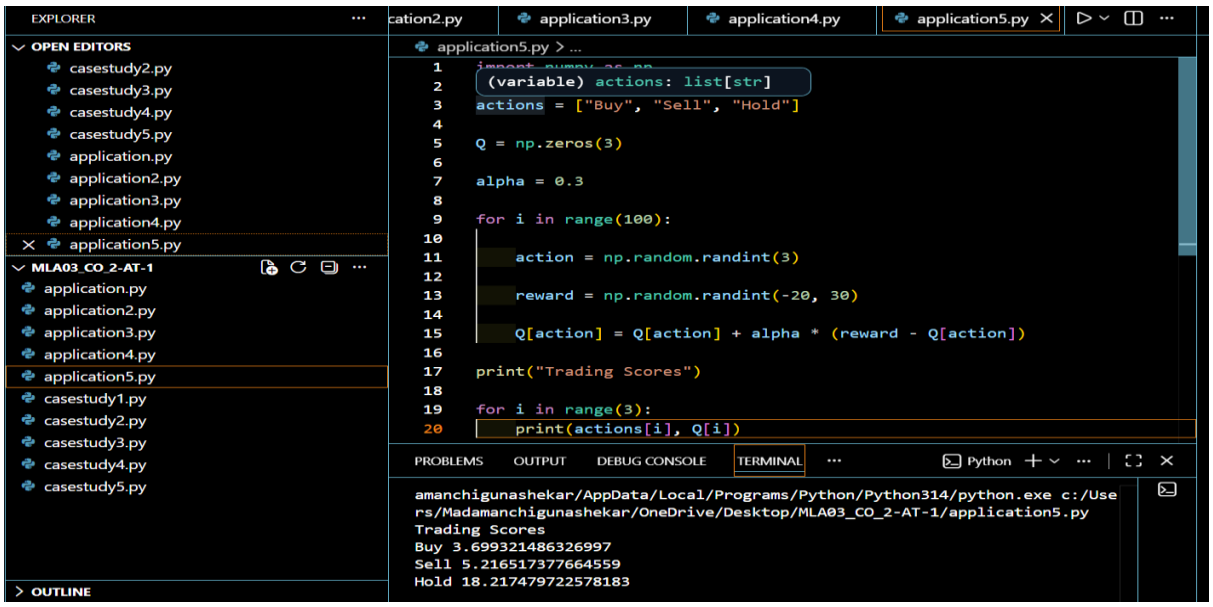
Algorithm

1. Initialize the Q-table.
2. Observe the grid state.
3. Select an action using the ε-greedy policy.
4. Execute the action.
5. Receive reward.
6. Update the Q-value.
7. Repeat until the optimal policy is learned.

Expected Output

- Efficient electricity distribution
- Reduced power loss
- Improved grid stability

CODE:-



Conclusion

An RL-based smart grid continuously adapts to changing demand and generation patterns, leading to efficient, reliable, and intelligent electricity distribution.